

BÁO CÁO BTVN 19/5

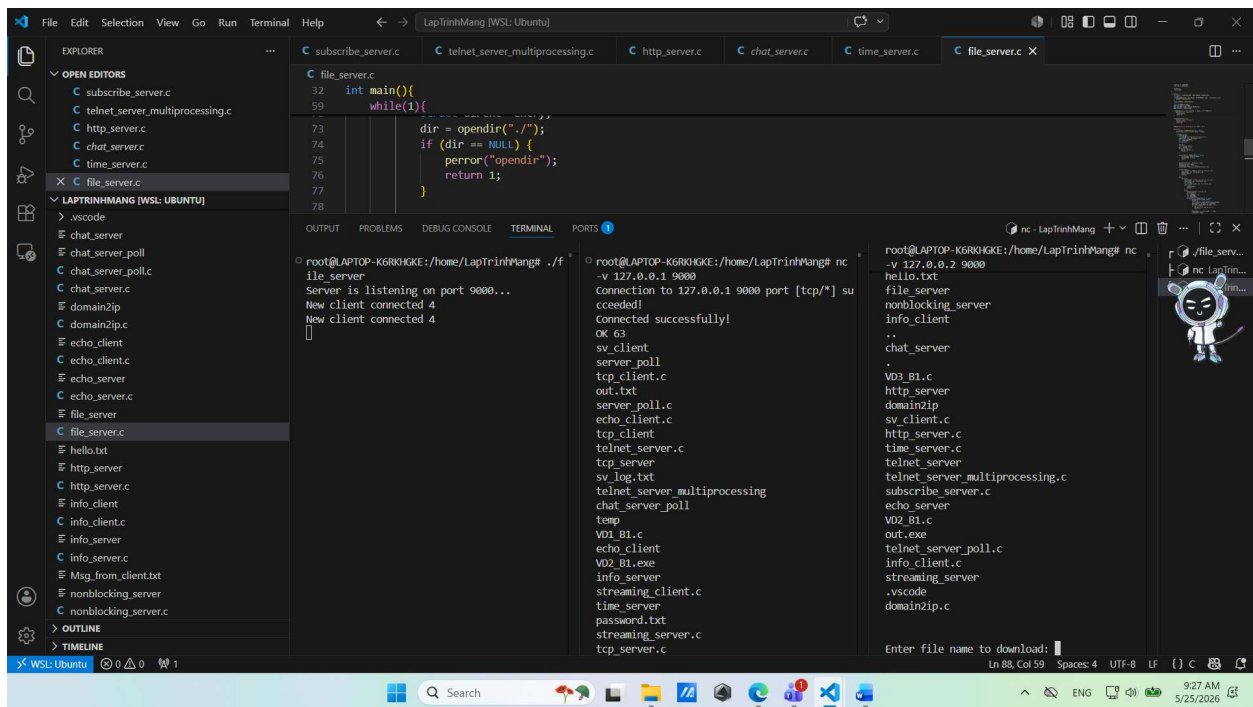
Bài 1:

Sử dụng kỹ thuật đa tiến trình, lập trình ứng dụng **file server** đơn giản như sau:

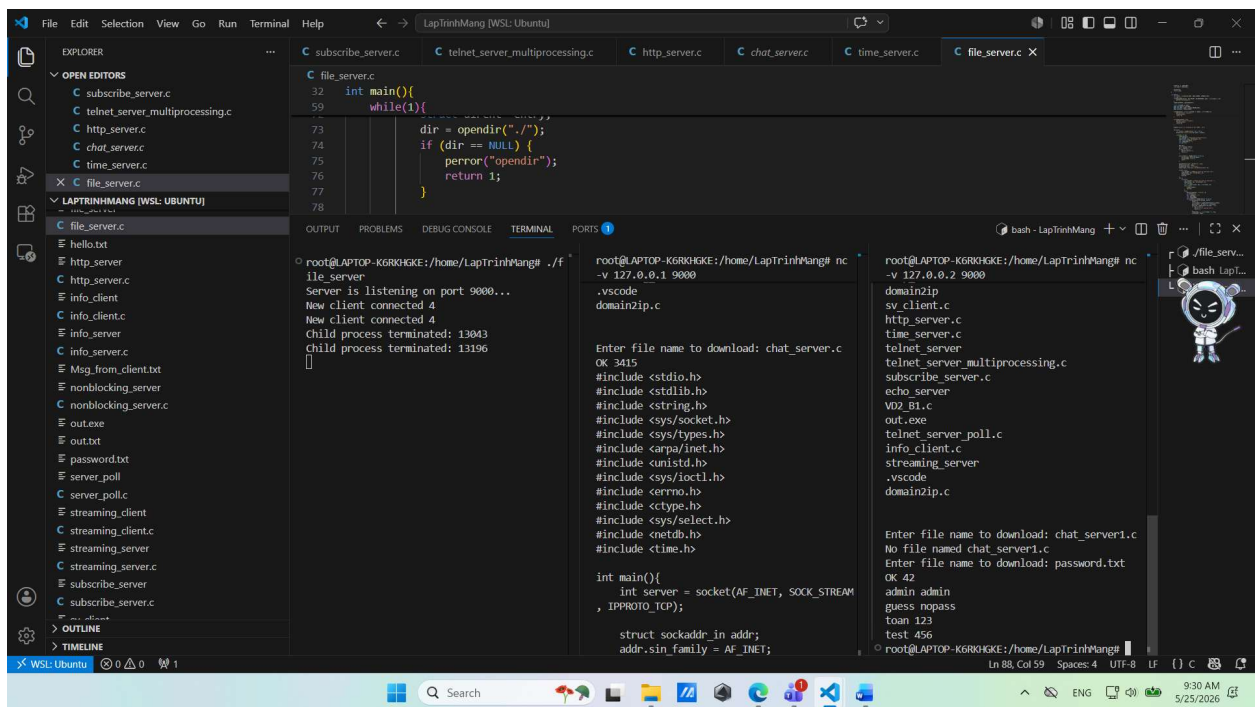
- Khi client mới được chấp nhận, server sẽ gửi danh sách các file cho client. Các file này trong thư mục được thiết lập trên server. Dòng đầu danh sách là chuỗi **"OK N\r\n"** nếu có N file trong thư mục. Các tên file phân cách nhau bởi ký tự **\r\n**, kết thúc danh sách bởi ký tự **\r\n\r\n**.
- Nếu không có file nào trong thư mục thì server gửi chuỗi **"ERROR No files to download\r\n"** rồi đóng kết nối.
- Client gửi tên file để tải về, server kiểm tra nếu file tồn tại thì gửi chuỗi **"OK N\r\n"** trong đó N là kích thước file, sau đó là nội dung file cho client và đóng kết nối, nếu file không tồn tại thì gửi thông báo lỗi và yêu cầu client gửi lại tên file.

Kết quả chạy thử:

- Chạy file `file_server.c` để tạo server lắng nghe kết nối và dùng netcat để tạo 2 client kết nối đến cổng 9000 của server, client sẽ nhận được thông báo kết nối thành công, số file có trong thư mục tải và danh sách các file có thể tải kèm thông báo nhập tên file muốn tải:



- Nếu client nhập tên file không tồn tại trong thư mục, server sẽ báo không tìm thấy file và yêu cầu nhập lại, nếu nhập đúng, server sẽ gửi thông báo kích thước file, nội dung file và sau đó đóng kết nối:



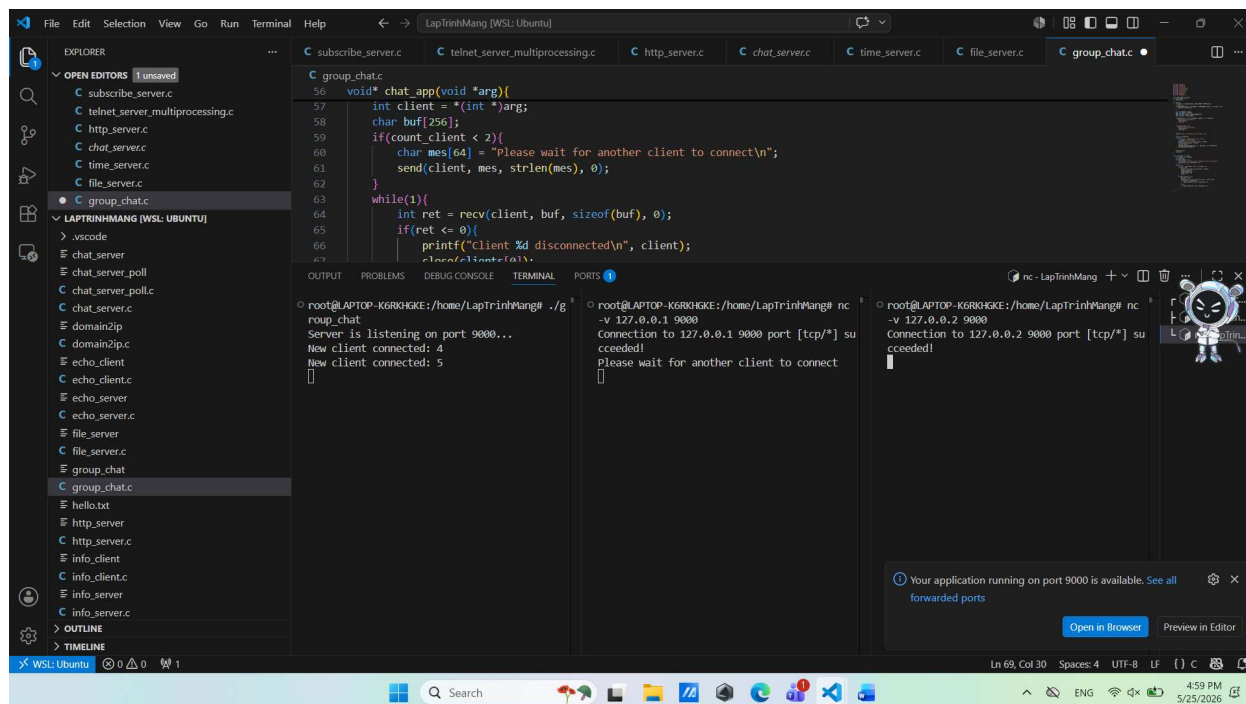
Bài 2:

Sử dụng kỹ thuật đa luồng, lập trình ứng dụng chat nhóm 2 người như sau:

- Mỗi client khi kết nối đến server sẽ được lưu vào hàng đợi, nếu số lượng client trong hàng đợi là 2 thì 2 client này sẽ được ghép cặp với nhau.
- Khi 2 client đã được ghép cặp thì tin nhắn gửi từ client này sẽ được chuyển tiếp sang client kia và ngược lại.
- Nếu 1 trong 2 client ngắt kết nối thì client còn lại cũng được ngắt kết nối.

Kết quả chạy thử:

- Chạy file group_chat.c và dùng netcat tạo 2 client kết nối đến server, khi client đầu tiên kết nối đến, server sẽ thông báo kết nối thành công và yêu cầu đợi client thứ 2 kết nối. Khi client thứ 2 kết nối đến, server thông báo kết nối thành công, cả 2 có thể tiến hành gửi tin nhắn:



The screenshot shows a Windows Subsystem for Linux (WSL) environment with Ubuntu. The terminal is divided into three panes. The left pane shows the file explorer with a list of files including group_chat.c, subscribe_server.c, telnet_server_multiprocessing.c, http_server.c, chat_server.c, time_server.c, file_server.c, and group_chat.c. The middle pane shows the code for group_chat.c, which implements a multi-threaded chat server. The right pane shows the output of the program, which includes the server listening on port 9000, receiving two clients, and displaying the message 'Please wait for another client to connect'.

```
group_chat.c
56 void* chat_app(void *arg){
57     int client = *(int *)arg;
58     char buf[256];
59     if(count_client < 2){
60         char mes[64] = "Please wait for another client to connect\n";
61         send(client, mes, strlen(mes), 0);
62     }
63     while(1){
64         int ret = recv(client, buf, sizeof(buf), 0);
65         if(ret <= 0){
66             printf("Client %d disconnected\n", client);
67             close(client);
68         }
69     }
70 }
```

```
root@LAPTOP-KGRHGKE:/home/LapTrinhHung# ./g
roup_chat
Server is listening on port 9000...
New client connected: 4
New client connected: 5
Please wait for another client to connect
```

```
root@LAPTOP-KGRHGKE:/home/LapTrinhHung# nc
-v 127.0.0.1 9000
Connection to 127.0.0.1 9000 port [tcp/*] su
cceded!
Please wait for another client to connect
```

```
root@LAPTOP-KGRHGKE:/home/LapTrinhHung# nc
-v 127.0.0.2 9000
Connection to 127.0.0.2 9000 port [tcp/*] su
cceded!
```

Your application running on port 9000 is available. See all forwarded ports

- Khi có client gửi tin nhắn, server hiện thông báo nhận được tin nhắn từ client và tiến hành gửi tin nhắn cho client còn lại:

The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying a project named 'LAPTRINH-MANG (WSL: UBUNTU)'. The file 'group_chat.c' is open in the editor. The code defines a chat application that listens on port 9000 and handles multiple clients. The terminal output shows the server listening on port 9000 and receiving messages from clients 4 and 5. The server responds to client 4 with 'Hello' and to client 5 with 'Hi'. The server also prints 'Client 4 disconnected' and 'Client 5 disconnected' when the clients disconnect.

```
void* chat_app(void *arg){
    int client = *(int *)arg;
    char buf[256];
    if(count_client < 2){
        char mes[64] = "Please wait for another client to connect\n";
        send(client, mes, strlen(mes), 0);
    }
    while(1){
        int ret = recv(client, buf, sizeof(buf), 0);
        if(ret <= 0){
            printf("Client %d disconnected\n", client);
            close(clientfd);
        }
    }
}
```

```
root@LAPTOP-K6RKHGKE:/home/LapTrinhMang# ./g
roup_chat
Server is listening on port 9000...
New client connected: 4
New client connected: 5
Receive from client 5: Hi
Receive from client 4: Hello
Receive from client 5: How are
Receive from client 5: you
Receive from client 4: I'm fine, and you
Receive from client 5: Good
[]
```

- Khi có 1 client ngắt kết nối, client kia cũng sẽ được ngắt kết nối:

The screenshot shows the Visual Studio Code editor with the file explorer on the left displaying a project named 'LAPTRINH-MANG (WSL: UBUNTU)'. The file 'group_chat.c' is open in the editor. The code defines a chat application that listens on port 9000 and handles multiple clients. The terminal output shows the server listening on port 9000 and receiving messages from clients 4 and 5. The server responds to client 4 with 'Hello' and to client 5 with 'Hi'. The server also prints 'Client 4 disconnected' and 'Client 5 disconnected' when the clients disconnect. The output shows that when client 4 disconnects, client 5 also disconnects.

```
void* chat_app(void *arg){
    int client = *(int *)arg;
    char buf[256];
    if(count_client < 2){
        char mes[64] = "Please wait for another client to connect\n";
        send(client, mes, strlen(mes), 0);
    }
    while(1){
        int ret = recv(client, buf, sizeof(buf), 0);
        if(ret <= 0){
            printf("Client %d disconnected\n", client);
            close(clientfd);
        }
    }
}
```

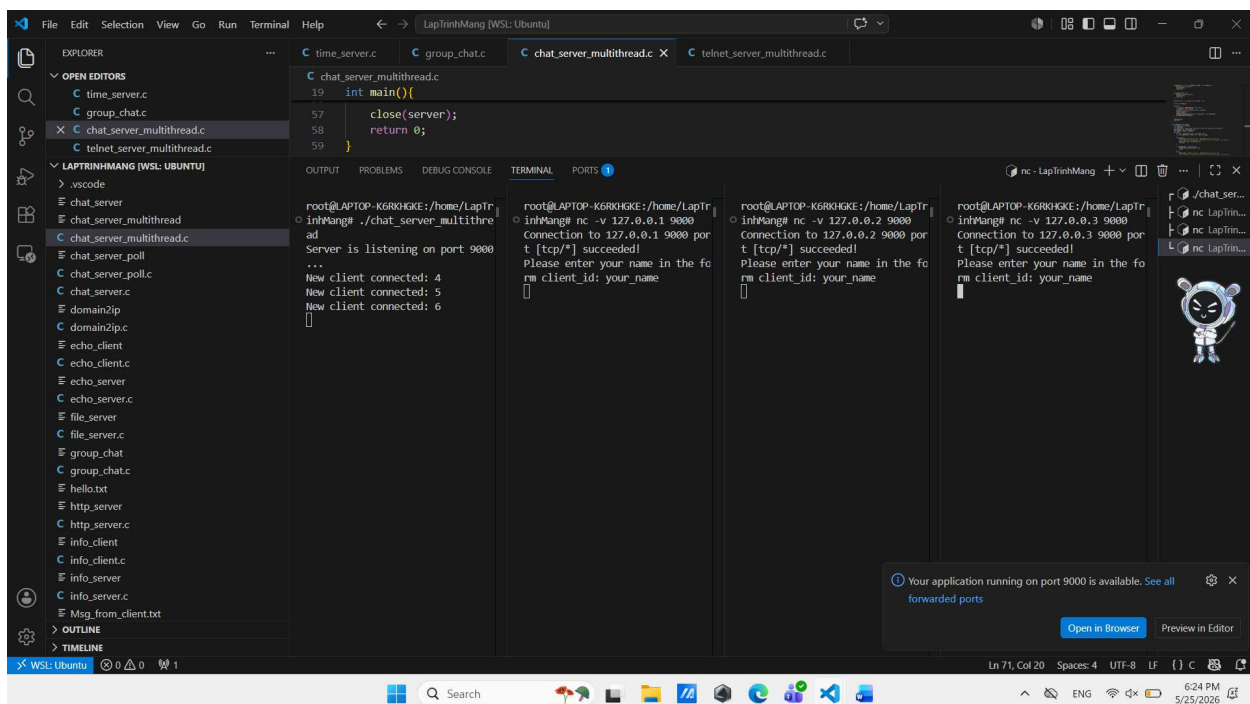
```
root@LAPTOP-K6RKHGKE:/home/LapTrinhMang# ./g
roup_chat
Server is listening on port 9000...
New client connected: 4
New client connected: 5
Receive from client 5: Hi
Receive from client 4: Hello
Receive from client 5: How are
Receive from client 5: you
Receive from client 4: I'm fine, and you
Receive from client 5: Good
[]
Client 5 disconnected
Receive from client 4: c
Client 4 disconnected
[]
```

Bài 3:

- Lập trình ứng dụng chat_server ([slide 173](#)) sử dụng kỹ thuật multithread.

Kết quả chạy thử:

- Chạy file chat_server_multithread và dùng netcat để tạo 3 client kết nối đến server, ở client thấy hiển thị thông báo kết nối thành công và nhận được yêu cầu nhập id:



```
File Edit Selection View Go Run Terminal Help
LapTrinhMang [WSL: Ubuntu]

EXPLORER
  chat_server.c
  group_chat.c
  chat_server_multithread.c
  telnet_server_multithread.c

LAPTRINHMANH [WSL: UBUNTU]
  .vscode
  chat_server
  chat_server_multithread
  chat_server_multithread.c
  chat_server_poll
  chat_server_poll.c
  chat_server.c
  domain2ip
  domain2ip.c
  echo_client
  echo_client.c
  echo_server
  echo_server.c
  file_server
  file_server.c
  group_chat
  group_chat.c
  hello.txt
  http_server
  http_server.c
  info_client
  info_client.c
  info_server
  info_server.c
  Msg_from_client.txt
  OUTLINE
  TIMELINE

chat_server_multithread.c
19 int main(){
57     close(server);
58     return 0;
59 }

OUTPUT PROBLEMS DEBUG CONSOLE TERMINAL PORTS
root@LAPTOP-K6RKHGKE:/home/LapTrinhMang# ./chat_server_multithread
Server is listening on port 9000
...
New client connected: 4
New client connected: 5
New client connected: 6

root@LAPTOP-K6RKHGKE:/home/LapTrinhMang# nc -v 127.0.0.1 9000
Connection to 127.0.0.1 9000 port [tcp/*] succeeded!
Please enter your name in the form client_id: your_name
[]

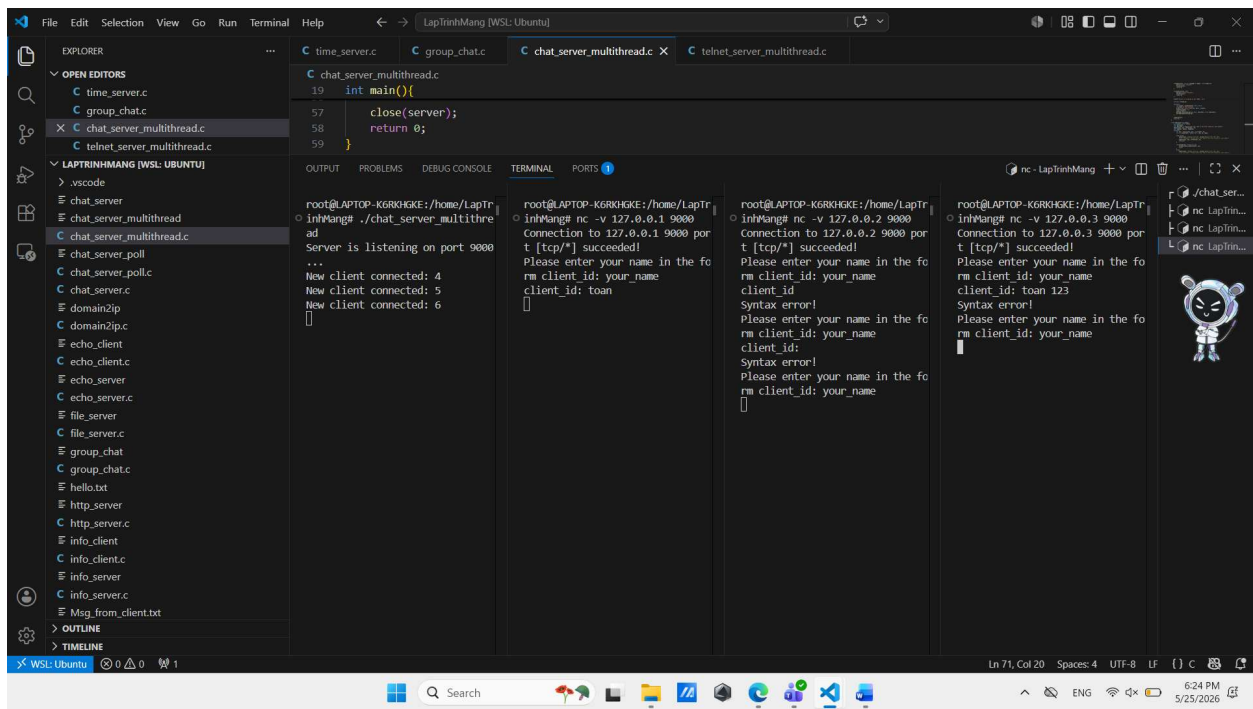
root@LAPTOP-K6RKHGKE:/home/LapTrinhMang# nc -v 127.0.0.2 9000
Connection to 127.0.0.2 9000 port [tcp/*] succeeded!
Please enter your name in the form client_id: your_name
[]

root@LAPTOP-K6RKHGKE:/home/LapTrinhMang# nc -v 127.0.0.3 9000
Connection to 127.0.0.3 9000 port [tcp/*] succeeded!
Please enter your name in the form client_id: your_name
[]

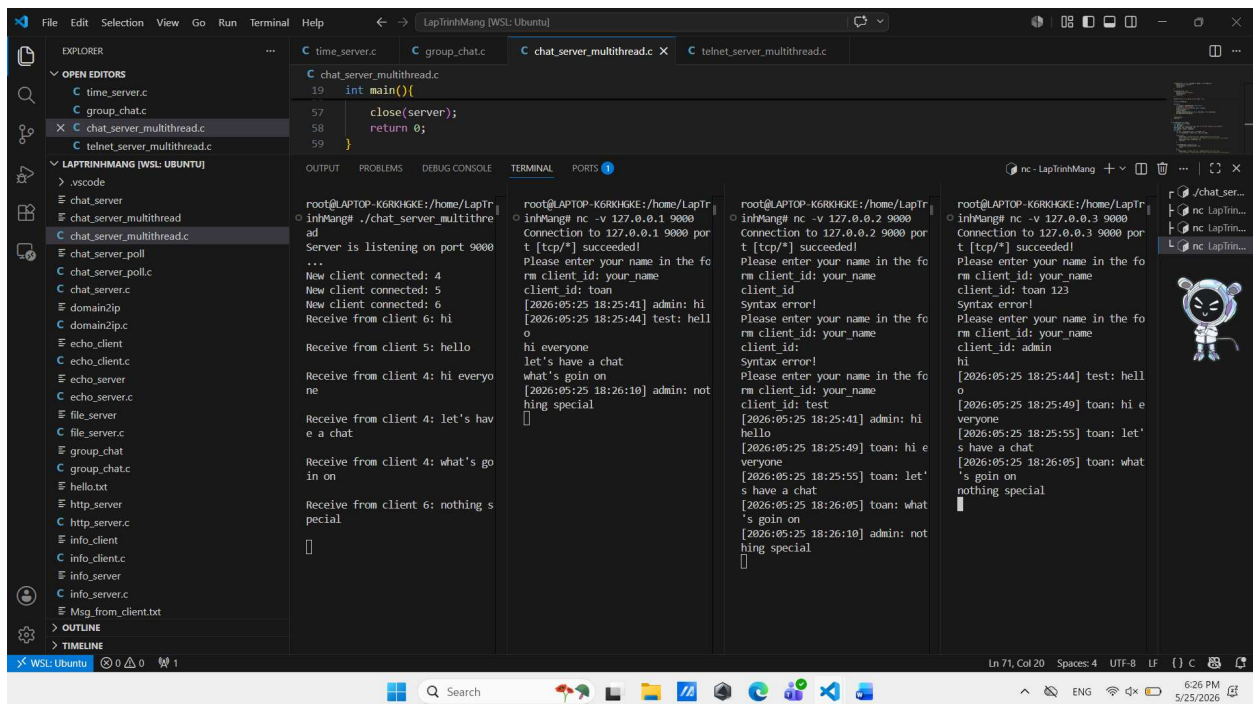
Your application running on port 9000 is available. See all forwarded ports
Open in Browser Preview in Editor

Ln 71, Col 20 Spaces: 4 UTF-8 LF {} C 6:24 PM 5/25/2026
```

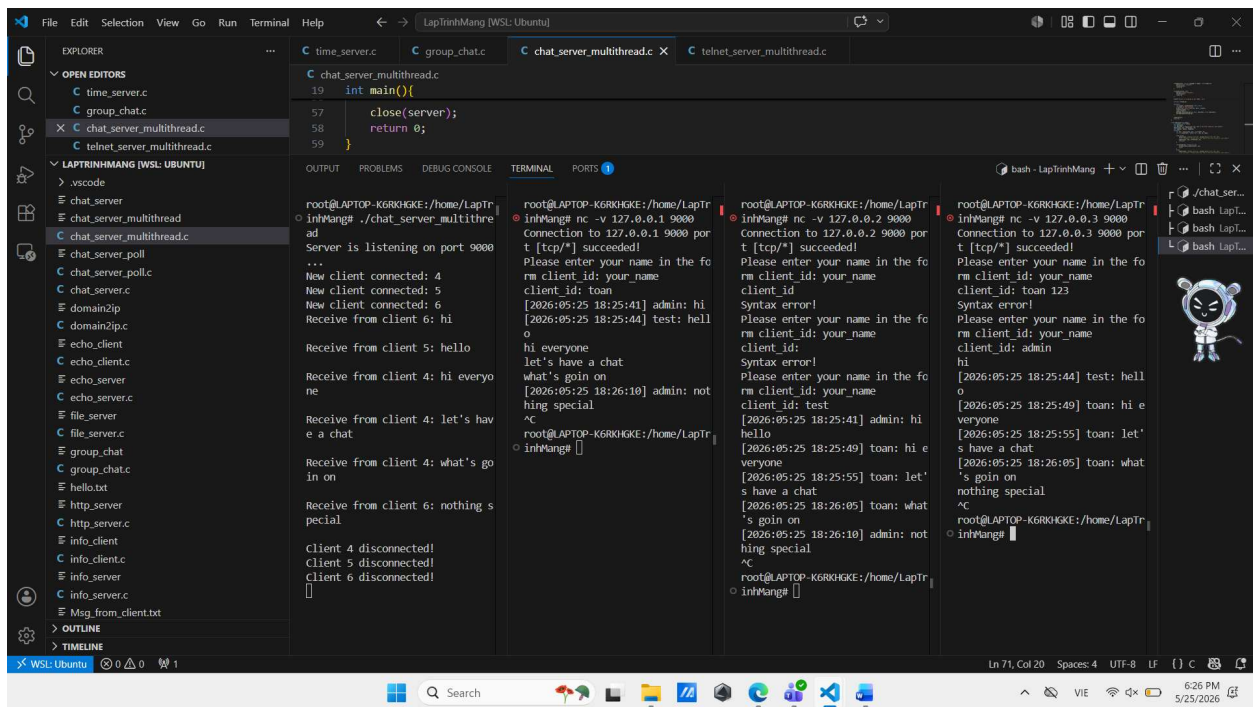
- Nếu nhập thừa hoặc thiếu tham số thì server sẽ thông báo syntax error và yêu cầu nhập lại, nếu nhập đúng thì client có thể tiến hành gửi tin nhắn:



- Khi server nhận được tin nhắn từ client thì sẽ tiến hành in tin nhắn ra và gửi tin nhắn đến các client còn lại kèm theo thời gian gửi tin nhắn:



- Thông báo khi client ngắt kết nối:

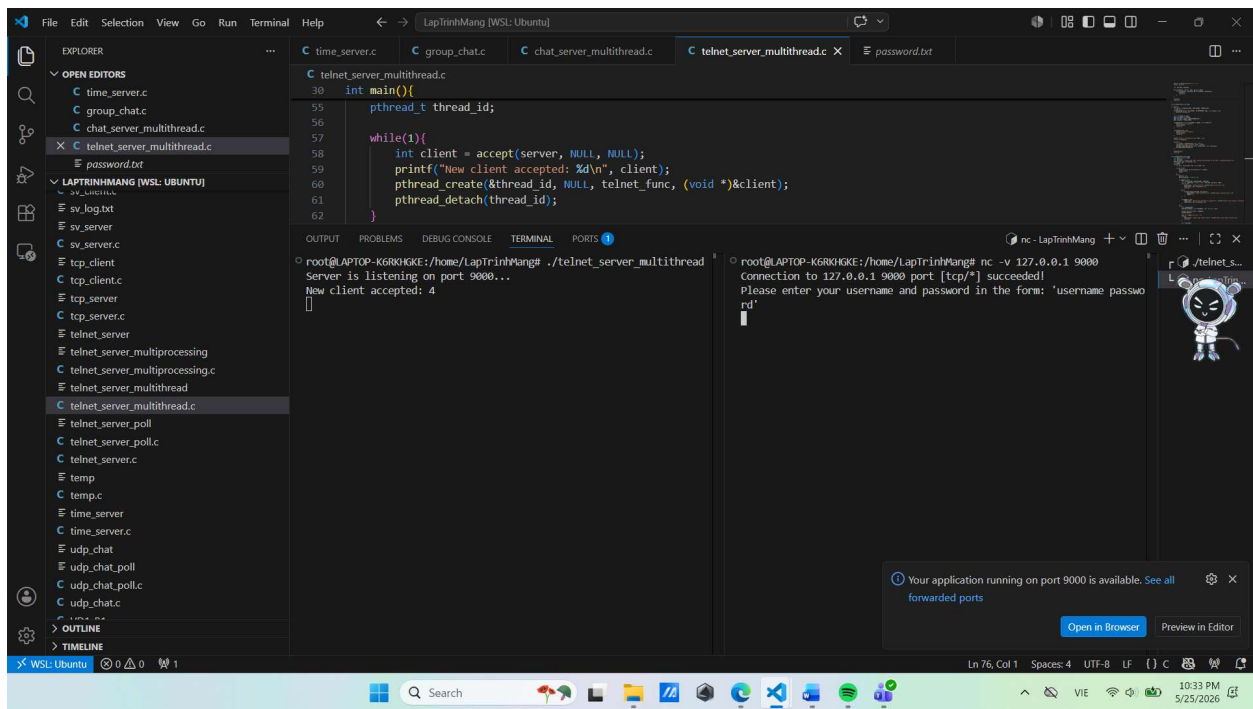


Bài 4:

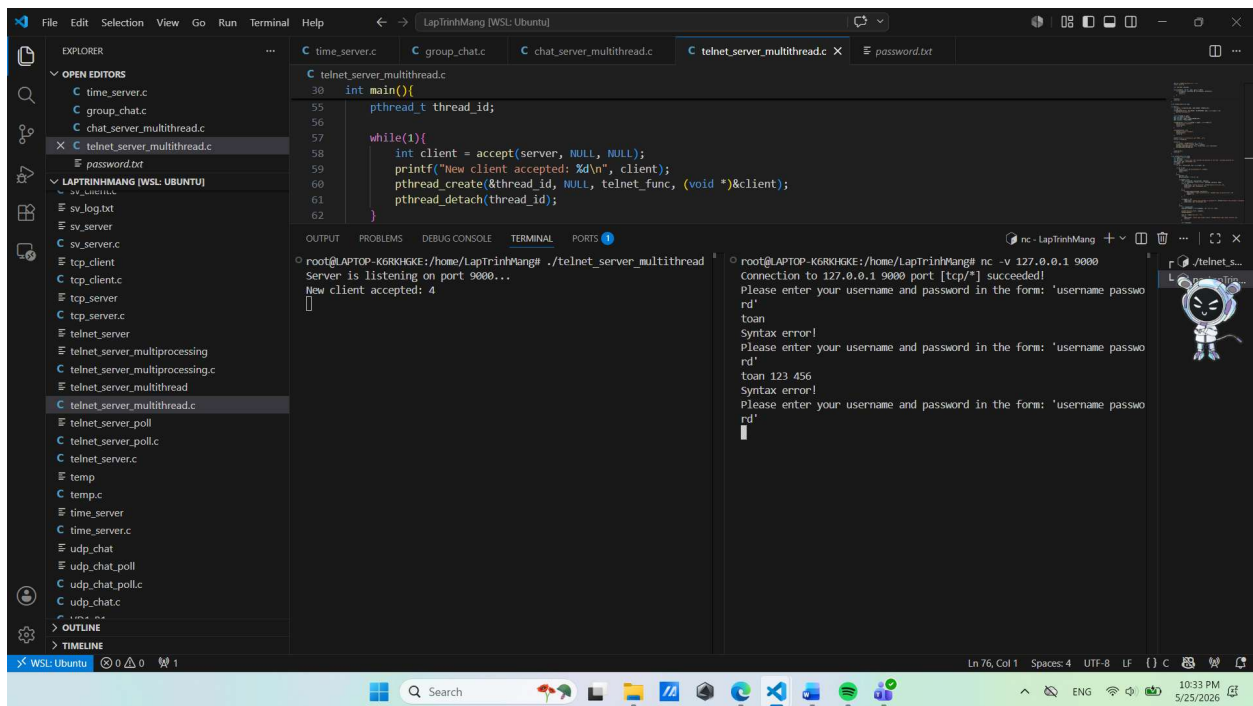
- Lập trình ứng dụng **telnet_server** ([slide 174](#)) sử dụng kỹ thuật multithread.

Kết quả chạy thử:

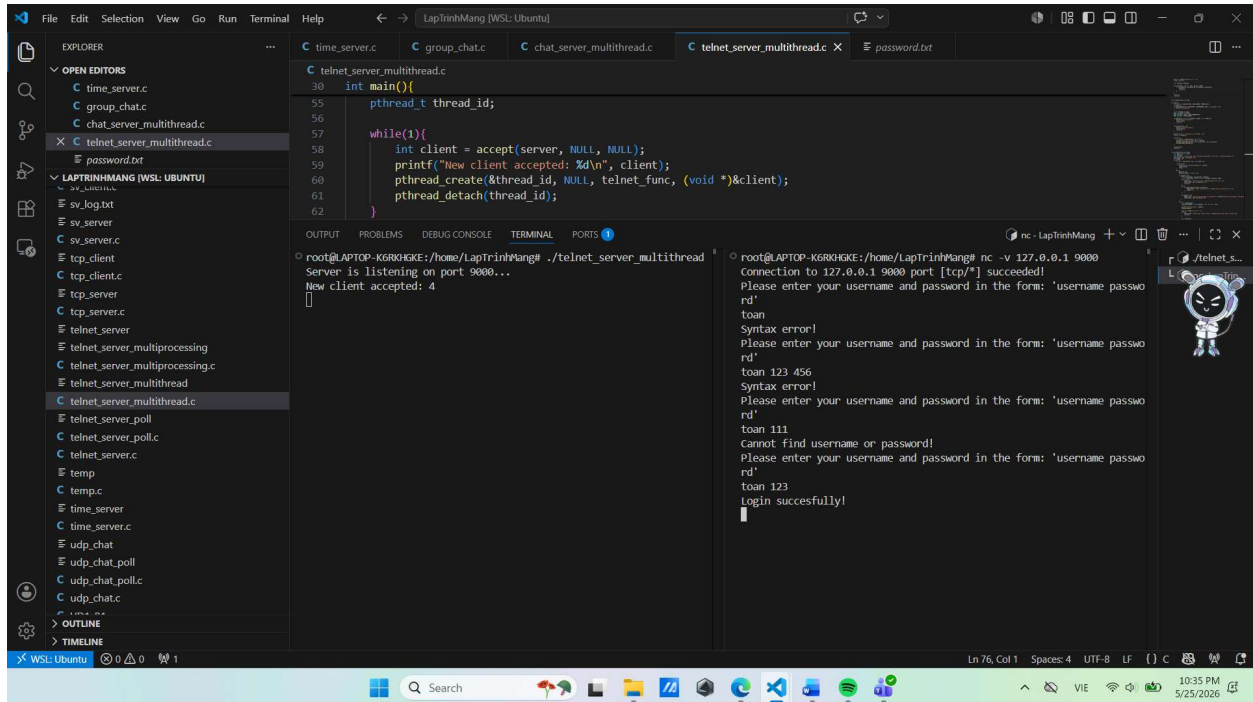
- Chạy file **telnet_server_multithread** và dùng netcat tạo client kết nối đến server thấy thông báo kết nối thành công và yêu cầu nhập tên đăng nhập và mật khẩu:



- Nếu nhập thừa hoặc thiếu tham số thì nhận được thông báo syntax error và được yêu cầu nhập lại:



- Nếu nhập sai tên đăng nhập hoặc mật khẩu thì được thông báo không tìm thấy tài khoản và được yêu cầu nhập lại, nếu nhập đúng thì được thông báo đăng nhập thành công và có thể tiến hành nhập lệnh:

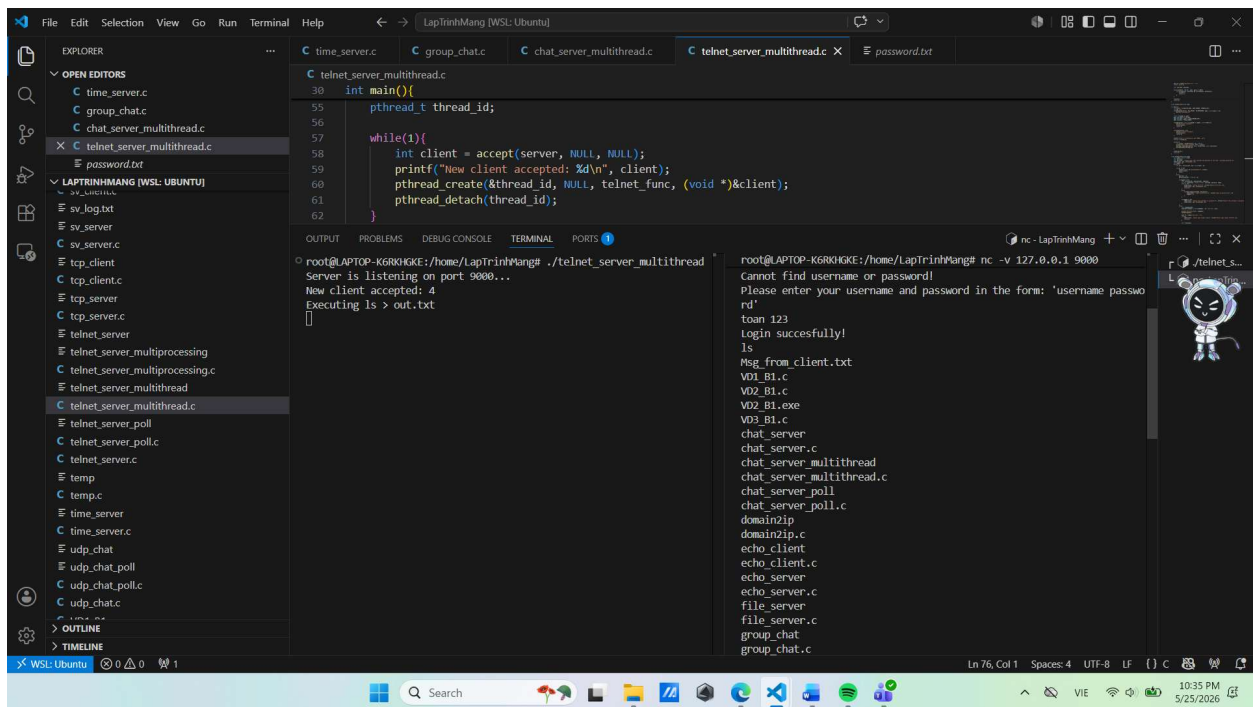


The screenshot shows a Visual Studio Code editor window titled 'LapTrinhMang (WSL: Ubuntu)'. The Explorer sidebar on the left lists several C files, with 'telnet_server_multithread.c' selected. The main editor displays the code for 'telnet_server_multithread.c', which is a multi-threaded server using pthreads. The code includes headers for `stdio.h`, `pthread.h`, and `unistd.h`. It defines a `telnet_func` that handles client connections and a `main` function that sets up the server and enters a loop to accept new clients.

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4
5 pthread_t thread_id;
6
7 int main() {
8     int client = accept(server, NULL, NULL);
9     printf("New client accepted: %d\n", client);
10    pthread_create(&thread_id, NULL, telnet_func, (void *)client);
11    pthread_detach(thread_id);
12 }
```

The OUTPUT pane shows the server's execution: 'Server is listening on port 9000...' and 'New client accepted: 4'. The TERMINAL pane shows a client connection using 'nc -v 127.0.0.1 9000'. The client sends a username 'toan' and a password '123', which are accepted by the server, resulting in the message 'Login successfully!'.

- Khi nhập lệnh, server in ra câu lệnh đang thực hiện, lưu vào file out.txt và trả kết quả về cho client:

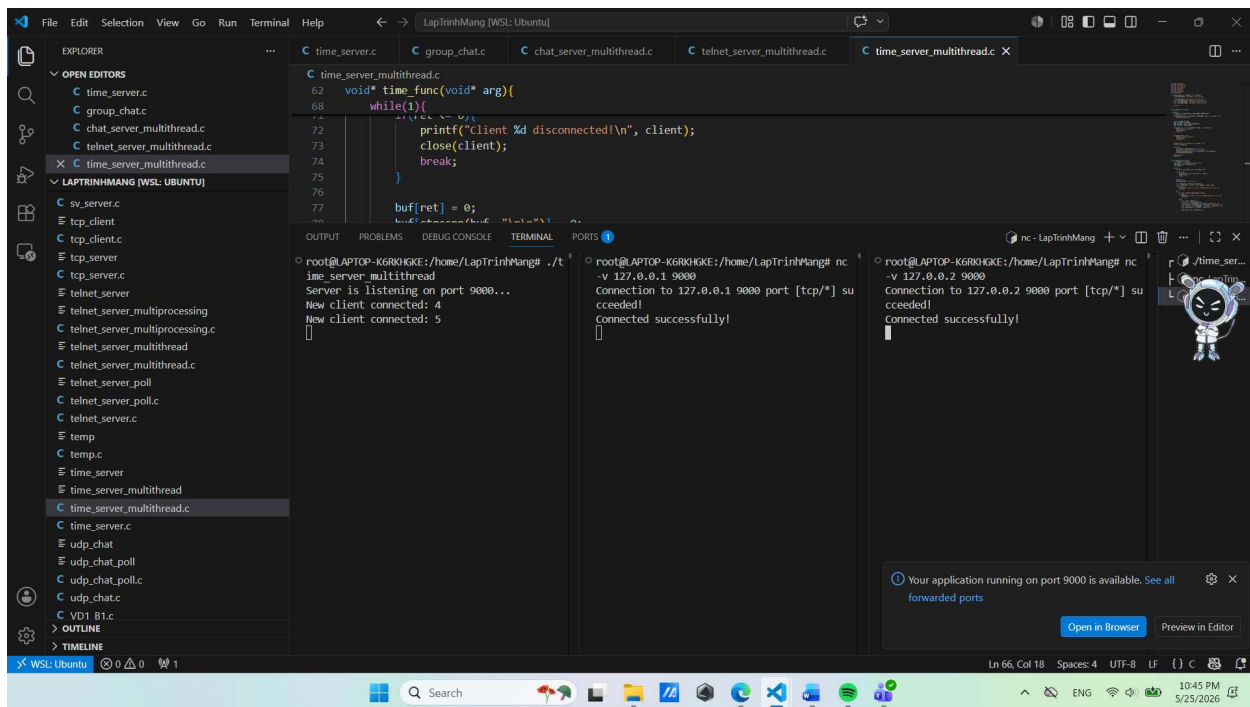


Bài 5:

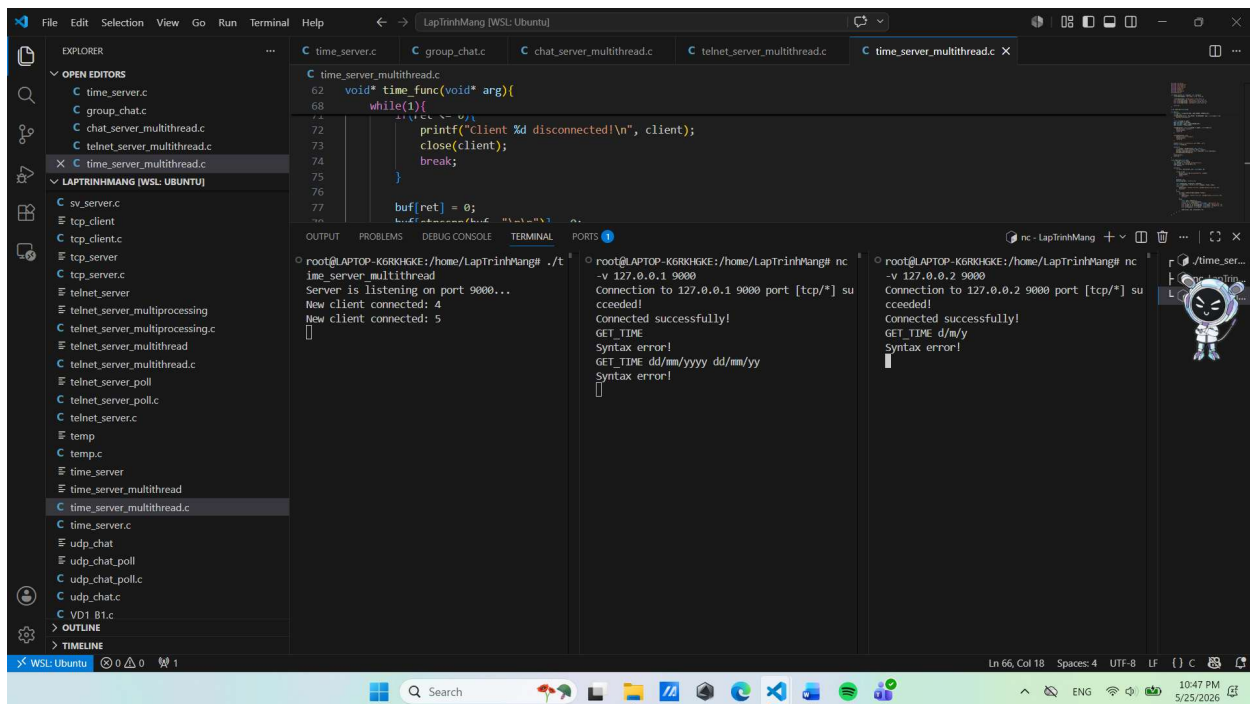
- Lập trình ứng dụng **time_server** ([slide 204](#)) sử dụng kỹ thuật multithread.

Kết quả chạy thử:

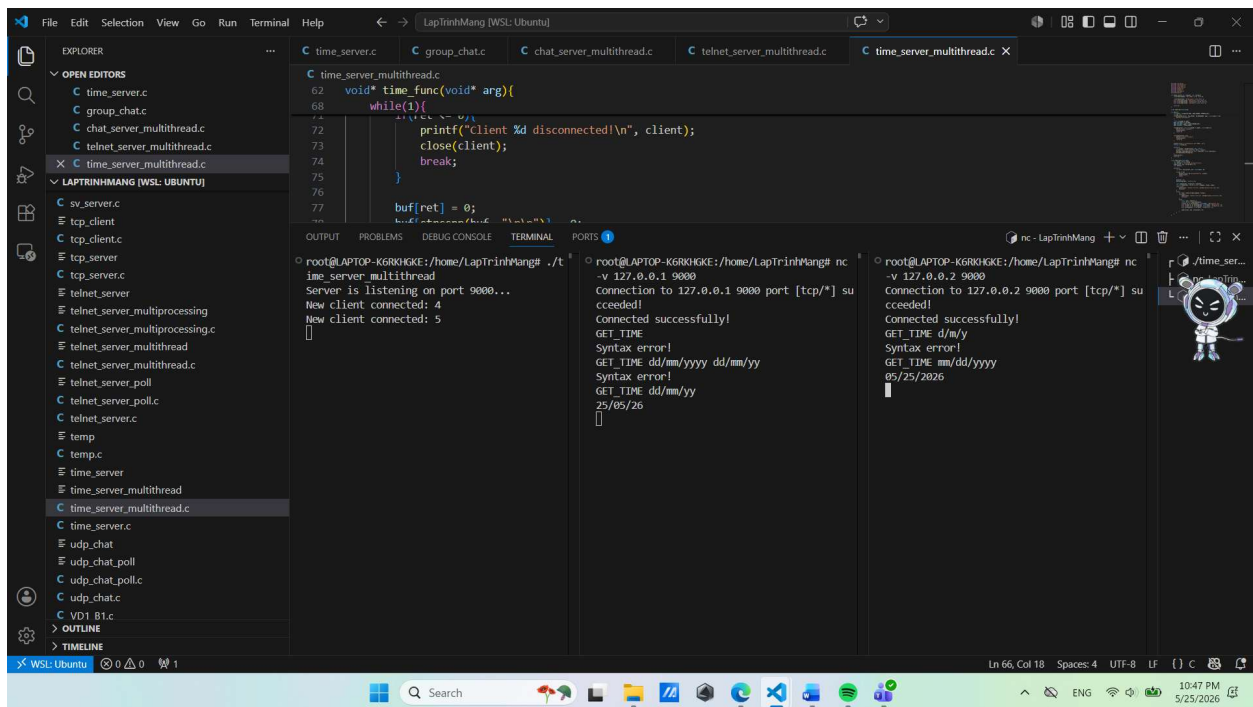
- Chạy file **time_server_multithread** và dùng netcat tạo client kết nối đến server, ở client nhận được thông báo kết nối thành công và có thể tiến hành nhập lệnh:



- Nếu nhập thừa/thiếu tham số hoặc sai định dạng thì sẽ được báo syntax error:



- Khi nhập đúng định dạng thì server sẽ trả lại kết quả thời gian hiện tại theo định dạng được yêu cầu:

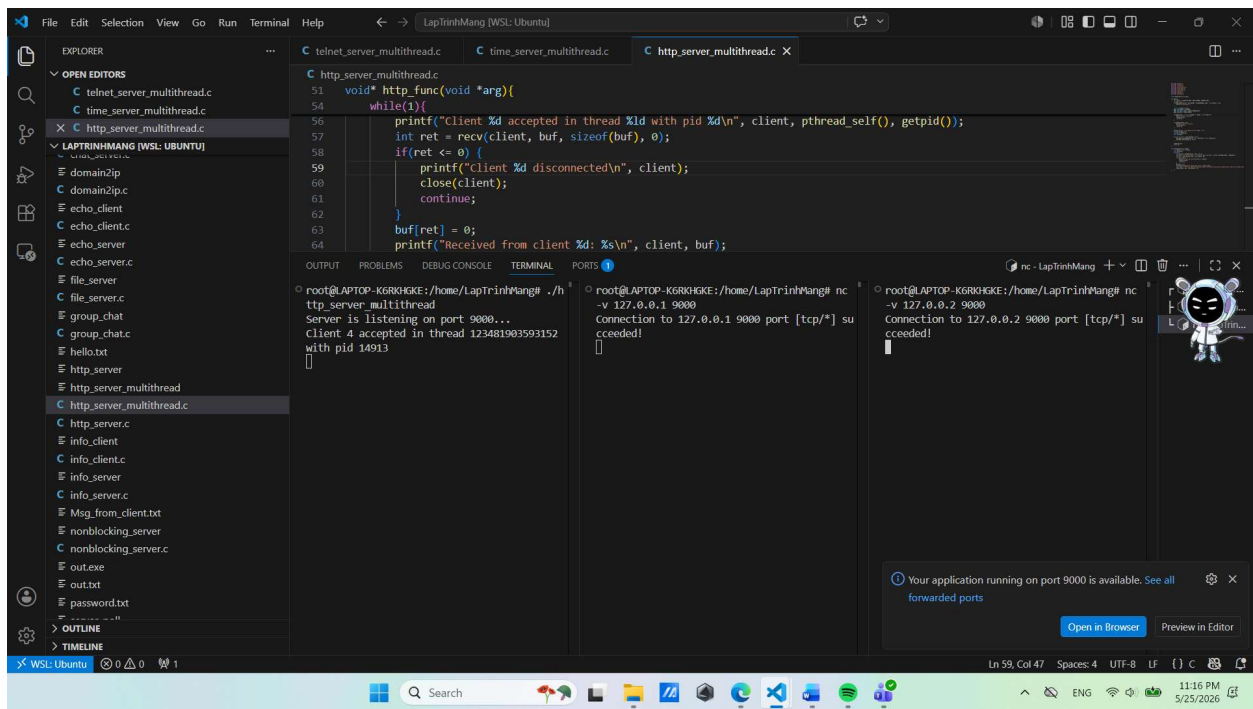


Bài 6:

- Lập trình ứng dụng **http_server** ([slide 154](#)) sử dụng kỹ thuật prethreading.

Kết quả chạy thử:

- Chạy file `http_server_multithread` và dùng netcat tạo client kết nối đến server thấy có thông báo kết nối thành công:



- Client gửi tin nhắn đến server và server phản hồi lại:

